

数据结构入门补充讲义

从操作矛盾出发选择结构

对 Day6 与补充材料的二周目台阶补充

这份补充的定位

- 栈、队列只快速复习，因为它们是你已有的地基。
- 二叉树、堆、二叉搜索树、并查集、区间结构都按重点讲。
- 目标不是把所有代码塞给你，而是让你知道每个结构为什么存在。

核心主线：先理解维护了什么信息，再谈怎么实现。

不要一上来猜模板

- 数据结构不是 STL 容器列表，而是一套维护信息的办法。
- 先看查询、修改和操作次数，再选择结构。
- 再问信息性质：能不能相减？能不能重复覆盖？能不能由左右合并？
- 工具最后才出现。哼，顺序错了就会把线段树写成大型玄学仪式。

做题先问五件事

- 数据是什么：数组、集合、树、图，还是动态元素？
- 查询什么：单点、前缀、区间、极值、连通性？
- 修改什么：无修改、单点修改、区间修改、合并集合？
- 操作次数多大：暴力会不会炸？
- 信息性质如何：可减、幂等、只可合并？

每个结构里存什么

1

存储信息

2

维护不变式

3

修改时更新受影响部分

4

查询时拼出答案

线性结构

线性结构快速复习

栈和队列不是重点难点，但它们提供了最基本的“处理顺序”模型。

栈：最近的未处理对象

- 规则：后进先出。
- 状态含义：还没有被匹配或消除的对象。
- 括号匹配中，栈里存的是未匹配的左括号。
- 遇到右括号，只能和最近的未匹配左括号尝试匹配。

栈的识别信号

- 最近出现的东西要最先处理。
- 括号、表达式、嵌套结构。
- 相邻消除，或者回退到上一个状态。

问自己：我是不是需要找最近的未处理元素？

队列：按时间顺序扩展

- 规则：先进先出。
- 状态含义：已经发现，但还没有扩展的点。
- BFS 中，先入队的点距离更短，所以先扩展。
- 第一次访问某个点时，通常就得到最短步数。

从线性结构走向树

- 栈和队列像一条线：每个元素主要看前后顺序。
- 更复杂的数据结构开始出现分叉：一个节点下面可以有多个后续位置。
- 树就是这种分叉关系的基础语言。
- 堆、二叉搜索树、线段树都借用了二叉树的骨架。

树基础

树：层级关系

树不是一条线，而是“父亲 - 儿子 - 子树”的层级结构。

有根树的基本词

- 根：整棵树的起点，没有父亲。
- 父亲和儿子：一条边连接的上下级关系。
- 叶子：没有儿子的节点。
- 深度：从根走到该节点经过的边数。
- 子树：以某个节点为根，连同下面所有后代形成的一整块。

树最重要的性质

- 除根以外，每个节点有且只有一个父亲。
- 从任意节点不断往父亲走，最终一定到根。
- 一个节点到根的路径唯一。
- 很多树上算法都在利用这个唯一父亲关系沉淀状态。

二叉树：最多两个儿子

- 二叉树每个节点最多有左儿子和右儿子。
- 左、右有区别，所以它比普通“最多两个孩子”的树更有结构。
- 二叉树天然适合递归：处理根，再处理左子树和右子树。
- 线段树、堆、BST 都可以从二叉树理解。

三种遍历

遍历	访问顺序	常见用途
前序	根 -> 左 -> 右	先处理当前节点，再下发任务
中序	左 -> 根 -> 右	BST 中会得到递增序列
后序	左 -> 右 -> 根	先算儿子，再合并到父亲

遍历的小例子



前序 : A B D E C

中序 : D B E A C

后序 : D E B C A

完全二叉树：可以放进数组

- 除最后一层外都填满，最后一层从左到右连续填。
- 形状规整，所以不一定需要指针。
- 使用 1 号下标时：左儿子是 $2x$ ，右儿子是 $2x + 1$ 。
- 父亲是 $x / 2$ 向下取整。

为什么这很重要

- 堆依赖完全二叉树的数组存储。
- 向上调整就是不断找父亲 $x / 2$ 。
- 向下调整就是比较 $2x$ 和 $2x + 1$ 两个儿子。
- 先懂这个骨架，堆才不会像黑盒。

堆与BST

堆：动态维护极值

堆解决的是“数据会变，但我每次要当前最大或最小”。

堆的不变式

- 小根堆：每个父亲都不大于它的儿子。
- 因此根节点一定是整棵树最小值。
- 堆不保证整体有序，只保证堆顶是极值。
- 这就是它比完整排序更轻的原因。

堆的插入

1

新元素放到末尾

2

保持完全二叉树形状

3

和父亲比较

4

不满足就向上交换

一次最多走树高 $O(\log n)$ 。

堆的删除堆顶

1

删除根节点

2

末尾元素补到根

3

和两个儿子比较

4

向下交换直到合法

排序一次 vs 堆维护

排序一次

- 数据不会再产生新元素
- 从头扫到尾即可
- 适合静态选择

堆维护

- 操作后会产生新元素
- 每次都要重新知道极值
- 适合动态极值

合并果子为什么用堆

- 每次取当前最小的两堆。
- 合并后产生一堆新的果子。
- 新堆还要参与之后的最小值选择。
- 静态排序一次不够，必须动态维护最小值。

堆与BST

二叉搜索树：有序的二叉树

BST 把 “小于往左、大于往右” 变成一条查找路径。

BST 的规则

- 左子树所有值都小于当前节点。
- 右子树所有值都大于当前节点。
- 每棵子树也必须满足同样规则。
- 中序遍历会得到递增序列。

BST 查找

1

从根开始

2

目标更小就往左

3

目标更大就往右

4

相等则找到

如果树平衡，每一步大约丢掉一半候选。

BST 插入

- 插入过程和查找几乎一样。
- 从根开始比较，小往左，大往右。
- 走到空位置时放入新节点。
- 插入后仍然保持中序递增。

BST 删除为什么麻烦

情况	处理方式	原因
叶子	直接删除	没有子树需要接回去
一个儿子	让儿子顶上来	有序关系仍可保持
两个儿子	用前驱或后继替换	要保住左右大小关系

普通 BST 会退化

理想情况

- 树比较平衡
- 高度约 $\log n$
- 查找、插入都快

退化情况

- 按 1,2,3,4,5 插入
- 树歪成一条链
- 查找退化成 $O(n)$

竞赛里常用 set/map

- 平衡树通过旋转保持高度 $O(\log n)$ 。
- `std::set` 维护无重复有序集合。
- `std::multiset` 允许重复元素。
- `std::map` 维护键到值的有序映射。

BST 的识别信号

- 动态插入、删除、查找。
- 需要有序遍历。
- 需要找前驱、后继。
- 需要维护键值映射。

并查集

并查集：集合归属

并查集不维护边的细节，只维护每个元素属于哪个集合。

并查集管理什么

- 它管理的是：谁和谁属于同一个集合。
- 每个集合选一个代表，也叫根。
- 判断同组：看两个元素的根是否相同。
- 合并集合：让一个根挂到另一个根下面。

为什么不直接搬集合

朴素做法

- 每次合并时搬动很多元素
- 一次合并可能 $O(n)$
- 多次操作容易炸

代表元做法

- 每个集合只认一个根
- 合并时只接两个根
- 查询时比较根

find: 找根

```
find(x):  
    if parent[x] == x:  
        return x  
    return find(parent[x])
```

根的父亲是自己，这是并查集的停止条件。

merge: 合并集合

```
merge(x, y):  
    root_x = find(x)  
    root_y = find(y)  
  
    if root_x != root_y:  
        parent[root_x] = root_y
```

并查集小模拟

操作	结构含义	结果
初始	每个点父亲都是自己	所有人各自一组
合并 1,2	1 的根接到 2 的根	1 和 2 同组
合并 3,4	3 的根接到 4 的根	3 和 4 同组
合并 2,3	两棵树接起来	1,2,3,4 同组

路径压缩不是魔法

- 如果 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, 查 1 的根要爬很久。
- 既然已经知道根是 4, 就把路上的父亲都改成 4。
- 这不会改变集合关系, 只是把树压扁。
- 下次查询同一批点时会更快。

路径压缩版 find

```
find(x):  
    if parent[x] != x:  
        parent[x] = find(parent[x])  
    return parent[x]
```

这行 `parent[x] = find(parent[x])` 就是在把路径压平。

按大小合并

- 如果总把大树挂到小树下面，树可能变高。
- 可以维护 $\text{size}[\text{root}]$ ，让小集合挂到大集合上。
- 路径压缩加按大小合并后，单次操作几乎可以看成 $O(1)$ 。
- 严格复杂度里有反阿克曼函数，入门阶段知道它极慢增长即可。

并查集擅长与不擅长

擅长

- 合并两个集合
- 判断是否同组
- Kruskal 判断加边是否成环

不擅长

- 列出集合所有成员
- 删除一条关系
- 直接维护集合内复杂信息

并查集识别信号

- 朋友的朋友也是朋友。
- 连通块合并。
- 判断两个点是否已经在同一集合。
- Kruskal 加边前判断是否成环。
- 等价关系具有传递性。

区间结构

区间结构：批量信息维护

区间结构解决的是数组上大量查询和修改的矛盾。

区间问题的基本矛盾

原数组

- 单点修改快
- 区间查询慢
- 每次可能扫 $O(n)$

前缀和

- 区间查询快
- 单点修改慢
- 修改会影响很多前缀

树状数组和线段树就是在这两端之间折中。

前缀和：无修改区间和

```
prefix[i] = a[1] + ... + a[i]
```

```
range_sum(l, r):
```

```
    return prefix[r] - prefix[l - 1]
```

依赖可减性：区间和能由两个前缀相减。

差分：只记录变化边界

```
add(l, r, c):  
    diff[l] += c  
    diff[r + 1] -= c  
  
restore:  
    a[i] = diff[1] + ... + diff[i]
```

适合离线区间加，最后一次性输出。

树状数组存什么

- $c[i]$ 存以 i 为右端点、长度为 $\text{lowbit}(i)$ 的一段和。
- $\text{prefix}(7)$ 可以拆成 $c[7] + c[6] + c[4]$ 。
- 查询时不断 $i -= \text{lowbit}(i)$ 。
- 单点加时，所有包含这个点的块都要更新： $i += \text{lowbit}(i)$ 。

lowbit 不是玄学

i	lowbit(i)	c[i] 管辖区间
1	1	[1, 1]
2	2	[1, 2]
3	1	[3, 3]
4	4	[1, 4]
6	2	[5, 6]
8	8	[1, 8]

树状数组伪代码

```
add(i, v):  
    while i <= n:  
        c[i] += v  
        i += lowbit(i)  
  
prefix(i):  
    ans = 0  
    while i > 0:  
        ans += c[i]  
        i -= lowbit(i)  
    return ans
```

ST 表：静态区间最值

- 最大值没有可减性，不能像前缀和那样相减。
- 但最大值有幂等性：重复覆盖一段不会影响答案。
- $f[k][i]$ 表示从 i 开始、长度为 2^k 的区间最值。
- 适合无修改、多次 RMQ 查询。

ST 表不适合有修改

适合

- 数组静态不变
- 查询最大值、最小值、gcd
- 预处理后 $O(1)$ 查询

不适合

- 中途单点修改
- 中途区间修改
- 需要动态维护答案

线段树：递归二分维护信息

- 根节点维护整个区间。
- 左右儿子分别维护左右半段。
- 父节点信息由两个儿子合并。
- 任意查询区间可以拆成少量线段树节点。

线段树和二叉树的关系

- 线段树的每个节点代表一个区间。
- 不是叶子的区间被分成左半和右半。
- 所以它天然形成一棵二叉树。
- 理解二叉树后，线段树就只是“节点里存区间信息”。

线段树查询三情况

- 当前节点完全在查询区间内：直接返回节点信息。
- 当前节点完全在查询区间外：忽略。
- 部分相交：递归左右儿子，再合并结果。
- 单点修改时，改叶子，然后一路回溯更新父亲。

懒标记：延迟更新

- 区间修改如果直接改所有叶子，可能退化成 $O(n)$ 。
- 完全覆盖一个节点时，先把这个节点的信息改正确。
- 再打一个标记：孩子以后被访问时要补上这次修改。
- 当前节点对外必须正确；孩子可以暂时旧，但访问前必须下传。

区间结构选择地图

问题形态	推荐结构	原因
无修改区间和	前缀和	可减, 查询 $O(1)$
离线区间加	差分	只记变化边界
单点改区间和	树状数组	轻量 $O(\log n)$
静态区间最值	ST 表	幂等, 查询 $O(1)$
区间改区间查	线段树	通用但代码更重

常见翻车点

- 只背 STL，不知道结构维护的状态含义。
- 二叉树遍历顺序混乱，不知道什么时候先根、什么时候后根。
- 堆误以为能保持完整有序，其实只保证堆顶极值。
- BST 忘记普通树会退化，需要平衡树或 STL。
- 并查集以为能删除关系或列出全体成员。
- 树状数组把 lowbit 当咒语，不知道 $c[i]$ 管哪一段。
- ST 表拿去做有修改的题。
- 线段树懒标记下传漏清空、漏改儿子信息。

最后的话

- 数据结构题的第一步是分类操作，不是猜模板。
- 每个结构都要问：存什么、管哪段、怎么更新、怎么查询。
- 线性结构解决顺序，树结构解决分叉与层级，集合结构解决归属，区间结构解决批量信息维护。

把查询类型、修改类型、操作次数、候选结构写出来。别偷懒。